

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Programación Distribuida en Java con RMI (Invocación Remota de Métodos)				
NÚMERO DE LA PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	09/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l4 SOLUCIÓN DE EJERCICIO RESUELTO Ejemplo de RMI en Java: Calculadora El archivo <code>Calculator.java</code> expone la interfaz remota del servicio de cálculo extendiendo la interfaz <code>java.rmi.Remote</code> y definiendo los métodos aritméticos fundamentales. El archivo <code>CalculatorImpl.java</code> proporciona la implementación concreta de la interfaz remota, heredando de la clase <code>UnicastRemoteObject</code> para permitir su exportación y el manejo transparente de la capa de referencia. El archivo <code>CalculatorServer.java</code> actúa como el programa anfitrión que inicializa un objeto de la calculadora y lo registra en el servicio RMI local mediante el método <code>Naming.rebind(...)</code>, poniéndolo a disposición de la red bajo el identificador respectivo. <pre>Naming.rebind("rmi://localhost:1099/CalculatorService", c); System.out.println("Calculator Service is ready.");</pre> El archivo <code>CalculatorClient.java</code> contiene la aplicación que se enlaza al registro de servicios para resolver y obtener una referencia al objeto remoto mediante el proceso de "lookup", invocando luego las rutinas de la calculadora y manejando cualquier excepción de conexión que pudiese surgir. <pre>Calculator c = (Calculator) Naming.lookup("rmi://localhost/CalculatorService"); System.out.println("The subtraction of " + num1 + " and " + num2 + " is: " + c.sub(num1, num2)); System.out.println("The addition of " + num1 + " and " + num2 + " is: " + c.add(num1, num2)); System.out.println("The multiplication of " + num1 + " and " + num2 + " is: " + c.mul(num1, num2)); System.out.println("The division of " + num1 + " and " + num2 + " is: " + c.div(num1, num2));</pre> A continuación, se muestra el proceso de compilación, levantamiento del registro RMI, ejecución del servidor y finalmente la prueba con múltiples clientes. Se observa que el cliente ejecuta las cuatro operaciones sobre los conjuntos de valores propuestos (7 y 1; 8 y 9).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
> javac *.java
> rmic CalculatorImpl
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.
> rmiregistry
^C
```

```
> java CalculatorServer
Calculator Service is ready.
^C
```

```
> java CalculatorClient 7 1
The subtraction of 7 and 1 is: 6
The addition of 7 and 1 is: 8
The multiplication of 7 and 1 is: 7
The division of 7 and 1 is: 7
```

```
> java CalculatorClient 8 9
The subtraction of 8 and 9 is: -1
The addition of 8 and 9 is: 17
The multiplication of 8 and 9 is: 72
The division of 8 and 9 is: 0
```

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Ejercicio 1: Sistema de Farmacia

El archivo `MedicineInterface.java` define el contrato para los objetos medicina exportados a nivel de red, estableciendo la semántica para consultar el inventario, adquirir dosis de un producto y obtener detalles formateados como cadenas de texto.

El archivo `Medicine.java` materializa esta interfaz proporcionando la lógica interna para administrar el estado del inventario de un fármaco, gestionando activamente las validaciones de suficiencia de existencias de modo que se arrojen instancias de `StockException` cuando un cliente solicita cantidades superiores al límite disponible.

```
public class Medicine extends UnicastRemoteObject implements MedicineInterface {
    private String name;
    private float unitPrice;
    private int stock;
    public Medicine() throws Exception {
        super();
    }
    public Medicine(String name, float price, int stock) throws Exception {
        super();
        this.name = name;
        unitPrice = price;
        this.stock = stock;
    }
    @Override
    public Medicine getMedicine(int amount) throws Exception {
        if (this.stock <= 0) throw new StockException("Stock vacío");
        if (this.stock - amount < 0) throw new StockException("Stock insuficiente");
        this.stock -= amount;
        Medicine aux = new Medicine(name, unitPrice * amount, stock);
        return aux;
    }
    @Override
    public int getStock() throws Exception {
        return this.stock;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```
@Override
public String print() throws Exception {
    return this.name + "\nPrecio: " + this.unitPrice + "\nStock: " + this.stock;
}
}
```

El archivo StockInterface.java y su implementación respectiva Stock.java estructuran el catálogo general de medicamentos utilizando un mapa HashMap que encapsula la base de datos de productos de salud para facilitar los procesos de búsqueda por nombres descriptivos a través del servicio en remoto.

```
public class Stock extends UnicastRemoteObject implements StockInterface {
    private HashMap<String, MedicineInterface> medicines = new HashMap<>();
    public Stock() throws RemoteException {
        super();
    }
    public void addMedicine(String name, float price, int stock) throws Exception {
        medicines.put(name, new Medicine(name, price, stock));
    }
    @Override
    public MedicineInterface buyMedicine(String name, int amount) throws Exception {
        MedicineInterface aux = medicines.get(name);
        if (aux == null) {
            throw new Exception("Imposible encontrar " + name);
        }
        MedicineInterface element = aux.getMedicine(amount);
        return element;
    }
    @Override
    public HashMap<String, MedicineInterface> getStockProducts() throws Exception {
        return this.medicines;
    }
}
```

El archivo ServerSide.java define el proceso demonio que arranca la instancia de la farmacia e inserta un conjunto inicial de objetos de tipo medicina como Paracetamol o Amoxilina dentro del catálogo antes de exponerlo públicamente en el registro.

```
public class ServerSide {
    public static void main(String[] args) throws Exception {
        Stock pharmacy = new Stock();
        pharmacy.addMedicine("Paracetamol", 3.2f, 10);
        pharmacy.addMedicine("Mejoral", 2.0f, 20);
        pharmacy.addMedicine("Amoxilina", 1.0f, 30);
        pharmacy.addMedicine("Aspirina", 5.0f, 40);
        Naming.rebind("rmi://localhost:1099/PHARMACY", pharmacy);
        System.out.println("Servidor listo");
    }
}
```

El archivo ClienteSide.java suministra la interfaz interactiva para el usuario final en la consola de comandos estándar, implementando un menú recursivo que recupera la colección del inventario para listar los ítems o bien ejecuta la orden directa de compra de insumos médicos restando stock de manera segura.

```
public class ClienteSide {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        StockInterface pharm = (StockInterface) Naming.lookup("rmi://localhost:1099/PHARMACY");
        System.out.println("Ingresa la opcion\n" + "1: Listar productos\n" + "2: Comprar Producto\n");
        int selection = sc.nextInt();
        if (selection == 1) {
            HashMap<String, MedicineInterface> aux = (HashMap<String, MedicineInterface>) pharm.getStockProducts();
            for (String key : aux.keySet()) {
                MedicineInterface e = (MedicineInterface) aux.get(key);
                System.out.println(e.print());
                System.out.println("-----*");
            }
        } else if (selection == 2) {
            System.out.println("Ingresa nombre de la medicina");
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```
String medicine = sc.next();
System.out.println("Ingrese cantidad a comprar");
int amount = sc.nextInt();
MedicineInterface aux = pharm.buyMedicine(medicine, amount);
System.out.println("Usted acaba de comprar");
System.out.println(aux.print());
} else {
    System.out.println("Seleccione una opcion valida");
}
}
sc.close();
}
```

A continuación, se muestra el proceso de compilación, generación de stubs y levantamiento del registro RMI. Posteriormente, el servidor administra el stock y los clientes interactúan mediante un menú listando los medicamentos disponibles y confirmando transacciones.

```
> javac *.java
> rmic Medicine Stock
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.
> rmiregistry
^C
```

```
> java ServerSide
Servidor listo
^C
```

```
> java ClienteSide
Ingresar la opcion
1: Listar productos
2: Comprar Producto

1
Aspirina
Precio: 5.0
Stock: 40
*-----*
Paracetamol
Precio: 3.2
Stock: 10
*-----*
Mejoral
Precio: 2.0
Stock: 20
*-----*
Amoxilina
Precio: 1.0
Stock: 30
*-----*
```

```
> java ClienteSide
Ingresar la opcion
1: Listar productos
2: Comprar Producto

2
Ingresar nombre de la medicina
Paracetamol
Ingresar cantidad a comprar
2
Usted acaba de comprar
Paracetamol
Precio: 6.4
Stock: 8
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Ejercicio 2: Sistema de Tarjetas de Crédito

El archivo `CreditCardInterface.java` expone de forma distribuida las primitivas financieras para validar compras, verificar saldo y mostrar el historial del plástico.

El archivo `CreditCardImpl.java` almacena el saldo acumulado en la tarjeta de crédito y la identidad del cliente, asegurando de forma persistente durante las transacciones que la cantidad acumulada no exceda el límite máximo del monto acreditado mediante validaciones internas controladas.

```
public class CreditCardImpl extends UnicastRemoteObject implements CreditCardInterface {
    private String cardNumber;
    private String ownerName;
    private double balance;
    private double creditLimit;

    public CreditCardImpl(String cardNumber, String ownerName, double creditLimit) throws Exception {
        super();
        this.cardNumber = cardNumber;
        this.ownerName = ownerName;
        this.creditLimit = creditLimit;
        this.balance = 0.0;
    }

    @Override
    public boolean processPayment(double amount) throws Exception {
        if (this.balance + amount <= this.creditLimit) {
            this.balance += amount;
            return true;
        }
        return false;
    }

    @Override
    public double getBalance() throws Exception {
        return this.balance;
    }

    @Override
    public String getCardDetails() throws Exception {
        return "Tarjeta: " + cardNumber + " | Titular: " + ownerName + " | Límite: " + creditLimit;
    }
}
```

El archivo `CreditServer.java` implementa el nodo central para el procesador de tarjetas que consolida el registro de crédito del cliente "Juan Perez" e inicia su vinculación global al demonio RMI bajo la identificación CREDITCARD dentro del puerto 1099 de la máquina local.

```
public class CreditServer {
    public static void main(String[] args) {
        try {
            CreditCardImpl card = new CreditCardImpl("1234-5678-9012-3456", "Juan Perez", 5000.00);
            Naming.rebind("rmi://localhost:1099/CREDITCARD", card);
            System.out.println("El servidor de tarjetas de crédito está listo.");
        } catch (Exception e) {
            System.out.println("Excepción del servidor: " + e);
        }
    }
}
```

El archivo `CreditClient.java` constituye el agente externo interactivo responsable de establecer contacto con el servicio del gestor de pagos para autorizar transferencias o rechazar gastos no cubiertos informando el saldo posterior a cada operación.

```
public class CreditClient {
    public static void main(String[] args) {
        try {
            CreditCardInterface card = (CreditCardInterface) Naming.lookup("rmi://localhost:1099/CREDITCARD");
            System.out.println("Conectado al servidor de tarjetas de crédito");
            System.out.println(card.getCardDetails());
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

Scanner sc = new Scanner(System.in);
System.out.println("Saldo actual: " + card.getBalance());
System.out.print("Ingrese el monto a cargar: ");
double amount = sc.nextDouble();

if (card.processPayment(amount)) {
    System.out.println("Pago procesado exitosamente.");
} else {
    System.out.println("Pago fallido: Límite de crédito excedido.");
}
System.out.println("Saldo actualizado: " + card.getBalance());
sc.close();
} catch (Exception e) {
    System.out.println("Excepción del cliente: " + e);
}
}
}

```

A continuación, se ilustran los comandos de preparación y lanzamiento de la arquitectura distribuida. Durante la evaluación interactiva, se autoriza la compra de 1500 unidades, pero el sistema rechaza exitosamente el siguiente cargo de 6000 debido a la rigurosa validación de límites.

```

> javac *.java
> rmic CreditCardImpl
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.
> rmiregistry
^C

```

```

> java CreditServer
El servidor de tarjetas de crédito está listo.
^C

```

```

> java CreditClient
Conectado al servidor de tarjetas de crédito
Tarjeta: 1234-5678-9012-3456 | Titular: Juan Perez | Limite: 5000.0
Saldo actual: 0.0
Ingrese el monto a cargar: 1500
Pago procesado exitosamente.
Saldo actualizado: 1500.0

```

```

> java CreditClient
Conectado al servidor de tarjetas de crédito
Tarjeta: 1234-5678-9012-3456 | Titular: Juan Perez | Limite: 5000.0
Saldo actual: 1500.0
Ingrese el monto a cargar: 6000
Pago fallido: Límite de crédito excedido.
Saldo actualizado: 1500.0

```

Ejercicio 3: Servicio de Conversión de Moneda

El archivo `CurrencyInterface.java` modela de forma abstracta las rutinas de cálculo financiero, proveyendo métodos específicos para la traslación de divisas como soles a euros o soles a dólares con retornos numéricos directos de doble precisión.

El archivo `CurrencyImpl.java` desarrolla la aplicación del modelo matemático con tasas de cambio predefinidas y estáticas a manera de propiedades constantes, realizando la conversión matemática mediante simples operaciones de división encapsuladas.

```

public class CurrencyImpl extends UnicastRemoteObject implements CurrencyInterface {
    private static final double TASA_DOLAR = 3.50;

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```
private static final double TASA_EURO = 4.10;

public CurrencyImpl() throws Exception {
    super();
}

@Override
public double convertirADolares(double monto) throws Exception {
    return monto / TASA_DOLAR;
}

@Override
public double convertirAEuros(double monto) throws Exception {
    return monto / TASA_EURO;
}
}
```

El archivo CurrencyServer.java funciona como el proceso inicializador que aloja y activa el servicio remoto bajo el nombre genérico de CURRENCY, manteniéndolo suspendido y operando de fondo para atender concurrentemente diversas peticiones cambiarias.

```
public class CurrencyServer {
    public static void main(String[] args) {
        try {
            CurrencyImpl converter = new CurrencyImpl();
            Naming.rebind("rmi://localhost:1099/CURRENCY", converter);
            System.out.println("El servidor conversor de monedas está listo.");
        } catch (Exception e) {
            System.out.println("Excepción del servidor: " + e);
        }
    }
}
```

El archivo CurrencyClient.java es el software interactivo que solicita el ingreso explícito de capital en soles desde el usuario y consulta al nodo principal RMI por la contraparte resultante en dólares o euros dependiendo del ítem del menú seleccionado.

```
public class CurrencyClient {
    public static void main(String[] args) {
        try {
            CurrencyInterface converter = (CurrencyInterface) Naming.lookup("rmi://localhost:1099/CURRENCY");
            System.out.println("Conectado al servidor conversor de monedas");

            Scanner sc = new Scanner(System.in);
            System.out.print("Ingrese monto en soles: ");
            double soles = sc.nextDouble();

            System.out.println("1. Convertir a Dolares");
            System.out.println("2. Convertir a Euros");
            System.out.print("Seleccione opcion: ");
            int opcion = sc.nextInt();

            if (opcion == 1) {
                System.out.printf("Monto en Dolares: %.2f\n", converter.convertirADolares(soles));
            } else if (opcion == 2) {
                System.out.printf("Monto en Euros: %.2f\n", converter.convertirAEuros(soles));
            } else {
                System.out.println("Opcion no valida");
            }
            sc.close();
        } catch (Exception e) {
            System.out.println("Excepción del cliente: " + e);
        }
    }
}
```

A continuación, se presenta la etapa de compilación RMI junto a la inicialización del servidor. Se efectúan interacciones desde el cliente comprobando de esta manera que el programa logra calcular montos en dólares (con un tipo de cambio de 3.50) y montos en euros (tipo de cambio 4.10) a partir de las bases en soles ingresadas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
> javac *.java
> rmic CurrencyImpl
Warning: generation and use of skeletons and static stubs for JRMP
is deprecated. Skeletons are unnecessary, and static stubs have
been superseded by dynamically generated stubs. Users are
encouraged to migrate away from using rmic to generate skeletons and static
stubs. See the documentation for java.rmi.server.UnicastRemoteObject.
> rmiregistry
^C
```

```
> java CurrencyServer
El servidor conversor de monedas está listo.
^C
```

```
> java CurrencyClient
Conectado al servidor conversor de monedas
Ingrese monto en soles: 38
1. Convertir a Dolares
2. Convertir a Euros
Seleccione opcion: 1
Monto en Dolares: 10.86
```

```
> java CurrencyClient
Conectado al servidor conversor de monedas
Ingrese monto en soles: 41
1. Convertir a Dolares
2. Convertir a Euros
Seleccione opcion: 2
Monto en Euros: 10.00
```

CUESTIONARIO

¿Cómo funciona el registro RMI?

El registro RMI funciona como un servicio de directorio a nivel de red que mapea identificadores de cadena (nombres lógicos) con las referencias a los objetos remotos correspondientes.

Actúa como un intermediario fundamental donde el servidor inscribe sus objetos utilizando métodos como `Naming.rebind()`, permitiendo que las aplicaciones cliente consulten este registro mediante `Naming.lookup()` para obtener los stubs necesarios para realizar las invocaciones de red.

¿Cuáles son las subclases para soportar carga dinámica de clases?

Para soportar la carga dinámica de clases en RMI, el entorno de ejecución de Java confía en un componente especializado denominado `RMIClassLoader`, el cual es una subclase interna de soporte que interactúa con las políticas de seguridad del sistema.

Este cargador permite descargar bajo demanda los bytes de las clases o stubs faltantes desde una URL específica provista por la propiedad `java.rmi.server.codebase`, facilitando la actualización distribuida sin requerir que los clientes tengan los archivos previamente compilados.

¿Qué ventajas y desventajas presenta RMI frente a la comunicación con sockets?

RMI abstrae la complejidad de la gestión explícita de los flujos de red y la serialización manual que requieren los sockets, permitiendo invocar métodos remotos con la misma semántica que los locales y reduciendo significativamente la longitud del código.

No obstante, presenta desventajas como la dependencia estricta del ecosistema Java en ambos extremos (sin usar CORBA), un rendimiento potencialmente inferior debido a la sobrecarga de la reflexión y la serialización, y complicaciones operativas al atravesar firewalls por la naturaleza de los puertos dinámicos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

¿Por qué RMI necesita que los objetos sean serializables y cómo se podría escalar este sistema a múltiples servidores RMI?

RMI exige que los objetos sean serializables debido a que necesita convertir su estado interno a un flujo de bytes secuencial para que pueda ser transmitido a través de la capa de transporte de la red hacia la máquina virtual remota.

Para escalar este sistema a múltiples servidores RMI, se puede implementar un balanceador de carga que distribuya las peticiones de registro, o utilizar tecnologías de clustering y un registro distribuido compartido como JNDI que gestione las referencias de múltiples nodos servidores.

¿Qué medidas de seguridad se deberían considerar para invocaciones remotas en entornos reales?

En entornos reales, las invocaciones remotas deben protegerse estableciendo túneles cifrados con RMI sobre SSL/TLS para prevenir la interceptación del tráfico en texto plano a nivel de infraestructura y conexiones vulnerables.

Asimismo, se debe configurar un SecurityManager estricto con un archivo de políticas restrictivo que limite los permisos, implementar mecanismos de autenticación antes de permitir la ejecución de los métodos, y encapsular el registro detrás de firewalls corporativos.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. La implementación de servicios distribuidos mediante RMI comprobó que la arquitectura de Invocación Remota de Métodos agiliza considerablemente el ciclo de desarrollo al aislar la lógica de serialización, el establecimiento de conexiones TCP y la gestión de flujos de datos.
2. El ejercicio del sistema de farmacia demostró que RMI soporta el paso de objetos complejos como parámetros y valores de retorno, permitiendo mantener la integridad de la orientación a objetos a través de los límites de las máquinas virtuales interconectadas en el proyecto de simulación.
3. Se verificó que el uso del RMI Registry es fundamental para desacoplar el ciclo de vida del servidor de la configuración del cliente, estableciendo un repositorio de nombres confiable y estandarizado donde los servicios quedan completamente accesibles con identificadores únicos y consistentes.

RECOMENDACIONES

1. Utilizar enfoques modernos basados en microservicios, como gRPC o API REST sobre HTTP/2, si se requiere interoperabilidad estricta con componentes y clientes construidos en lenguajes distintos a la máquina virtual del sistema Java utilizado inicialmente.
2. Centralizar la configuración de la red (puertos, hosts y propiedades de seguridad) en archivos de propiedades externos para evitar tener que recompilar el código fuente cuando la infraestructura de despliegue se modifique y escale sobre la marcha.
3. Implementar rutinas de manejo exhaustivo de RemoteException en el lado del cliente con estrategias de retardo exponencial (exponential backoff) para asegurar la recuperación transparente frente a interrupciones ocasionales, reintentando automáticamente en caso de problemas técnicos.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México: Pearson Educación.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- [4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
- [5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

ANEXOS

Ejercicio 1: Sistema de Farmacia

Interfaz de Medicina

```
import java.rmi.Remote;

public interface MedicineInterface extends Remote {
    public Medicine getMedicine(int amount) throws Exception;
    public int getStock() throws Exception;
    public String print() throws Exception;
}
```

Implementación de Medicina

```
import java.rmi.server.UnicastRemoteObject;

public class Medicine extends UnicastRemoteObject implements MedicineInterface {
    private String name;
    private float unitPrice;
    private int stock;

    public Medicine() throws Exception {
        super();
    }

    public Medicine(String name, float price, int stock) throws Exception {
        super();
        this.name = name;
        unitPrice = price;
        this.stock = stock;
    }

    @Override
    public Medicine getMedicine(int amount) throws Exception {
        if (this.stock <= 0) throw new StockException("Stock vacío");
        if (this.stock - amount < 0) throw new StockException("Stock insuficiente");
        this.stock -= amount;
        Medicine aux = new Medicine(name, unitPrice * amount, stock);
        return aux;
    }

    @Override
    public int getStock() throws Exception {
        return this.stock;
    }

    @Override
    public String print() throws Exception {
        return this.name + "\nPrecio: " + this.unitPrice + "\nStock: " + this.stock;
    }
}
```

Interfaz de Stock

```
import java.rmi.Remote;
import java.util.HashMap;

public interface StockInterface extends Remote {
    public HashMap<String, MedicineInterface> getStockProducts() throws Exception;
    public void addMedicine(String name, float price, int stock) throws Exception;
    public MedicineInterface buyMedicine(String name, int amount) throws Exception;
}
```

Implementación de Stock

```
import java.util.HashMap;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class Stock extends UnicastRemoteObject implements StockInterface {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```
private HashMap<String, MedicineInterface> medicines = new HashMap<>();
public Stock() throws RemoteException {
    super();
}
public void addMedicine(String name, float price, int stock) throws Exception {
    medicines.put(name, new Medicine(name, price, stock));
}
@Override
public MedicineInterface buyMedicine(String name, int amount) throws Exception {
    MedicineInterface aux = medicines.get(name);
    if (aux == null) {
        throw new Exception("Imposible encontrar " + name);
    }
    MedicineInterface element = aux.getMedicine(amount);
    return element;
}
@Override
public HashMap<String, MedicineInterface> getStockProducts() throws Exception {
    return this.medicines;
}
}
```

Excepción de Stock

```
public class StockException extends Exception {
    public StockException(String msg) {
        super(msg);
    }
}
```

Servidor de Farmacia

```
import java.rmi.*;
public class ServerSide {
    public static void main(String[] args) throws Exception {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        Stock pharmacy = new Stock();
        pharmacy.addMedicine("Paracetamol", 3.2f, 10);
        pharmacy.addMedicine("Mejoral", 2.0f, 20);
        pharmacy.addMedicine("Amoxilina", 1.0f, 30);
        pharmacy.addMedicine("Aspirina", 5.0f, 40);
        Naming.rebind("rmi://localhost:1099/PHARMACY", pharmacy);
        System.out.println("Servidor listo");
    }
}
```

Cliente de Farmacia

```
import java.rmi.Naming;
import java.util.HashMap;
import java.util.Scanner;
public class ClienteSide {
    public static void main(String[] args) throws Exception {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        Scanner sc = new Scanner(System.in);
        StockInterface pharm = (StockInterface) Naming.lookup("rmi://localhost:1099/PHARMACY");
        System.out.println("Ingresa la opcion\n" + "1: Listar productos\n" + "2: Comprar Producto\n");
        int selection = sc.nextInt();
        if (selection == 1) {
            HashMap<String, MedicineInterface> aux = (HashMap<String, MedicineInterface>) pharm.getStockProducts();
            for (String key : aux.keySet()) {
                MedicineInterface e = (MedicineInterface) aux.get(key);
                System.out.println(e.print());
                System.out.println("*.-----*");
            }
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

    }
} else if (selection == 2) {
    System.out.println("Ingrese nombre de la medicina");
    String medicine = sc.next();
    System.out.println("Ingrese cantidad a comprar");
    int amount = sc.nextInt();
    MedicineInterface aux = pharm.buyMedicine(medicine, amount);
    System.out.println("Usted acaba de comprar");
    System.out.println(aux.print());
} else {
    System.out.println("Seleccione una opcion valida");
}
}
sc.close();
}
}

```

Ejercicio 2: Sistema de Tarjetas de Crédito

Interfaz de Tarjeta de Crédito

```

import java.rmi.Remote;

public interface CreditCardInterface extends Remote {
    public boolean processPayment(double amount) throws Exception;
    public double getBalance() throws Exception;
    public String getCardDetails() throws Exception;
}

```

Implementación de Tarjeta de Crédito

```

import java.rmi.server.UnicastRemoteObject;

public class CreditCardImpl extends UnicastRemoteObject implements CreditCardInterface {
    private String cardNumber;
    private String ownerName;
    private double balance;
    private double creditLimit;

    public CreditCardImpl(String cardNumber, String ownerName, double creditLimit) throws Exception {
        super();
        this.cardNumber = cardNumber;
        this.ownerName = ownerName;
        this.creditLimit = creditLimit;
        this.balance = 0.0;
    }

    @Override
    public boolean processPayment(double amount) throws Exception {
        if (this.balance + amount <= this.creditLimit) {
            this.balance += amount;
            return true;
        }
        return false;
    }

    @Override
    public double getBalance() throws Exception {
        return this.balance;
    }

    @Override
    public String getCardDetails() throws Exception {
        return "Tarjeta: " + cardNumber + " | Titular: " + ownerName + " | Límite: " + creditLimit;
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

Servidor de Tarjetas de Crédito

```
import java.rmi.Naming;
public class CreditServer {
    public static void main(String[] args) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        try {
            CreditCardImpl card = new CreditCardImpl("1234-5678-9012-3456", "Juan Perez", 5000.00);
            Naming.rebind("rmi://localhost:1099/CREDITCARD", card);
            System.out.println("El servidor de tarjetas de crédito está listo.");
        } catch (Exception e) {
            System.out.println("Excepción del servidor: " + e);
        }
    }
}
```

Cliente de Tarjetas de Crédito

```
import java.rmi.Naming;
import java.util.Scanner;
public class CreditClient {
    public static void main(String[] args) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        try {
            CreditCardInterface card = (CreditCardInterface) Naming.lookup("rmi://localhost:1099/CREDITCARD");
            System.out.println("Conectado al servidor de tarjetas de crédito");
            System.out.println(card.getCardDetails());

            Scanner sc = new Scanner(System.in);
            System.out.println("Saldo actual: " + card.getBalance());
            System.out.print("Ingrese el monto a cargar: ");
            double amount = sc.nextDouble();

            if (card.processPayment(amount)) {
                System.out.println("Pago procesado exitosamente.");
            } else {
                System.out.println("Pago fallido: Límite de crédito excedido.");
            }
            System.out.println("Saldo actualizado: " + card.getBalance());
            sc.close();
        } catch (Exception e) {
            System.out.println("Excepción del cliente: " + e);
        }
    }
}
```

Ejercicio 3: Servicio de Conversión de Moneda

Interfaz de Conversión de Moneda

```
import java.rmi.Remote;
public interface CurrencyInterface extends Remote {
    public double convertirADolares(double monto) throws Exception;
    public double convertirAEuros(double monto) throws Exception;
}
```

Implementación de Conversión de Moneda

```
import java.rmi.server.UnicastRemoteObject;
public class CurrencyImpl extends UnicastRemoteObject implements CurrencyInterface {
    private static final double TASA_DOLAR = 3.50;
    private static final double TASA_EURO = 4.10;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

public CurrencyImpl() throws Exception {
    super();
}

@Override
public double convertirADolares(double monto) throws Exception {
    return monto / TASA_DOLAR;
}

@Override
public double convertirAEuros(double monto) throws Exception {
    return monto / TASA_EURO;
}
}

```

Servidor de Conversión de Moneda

```

import java.rmi.Naming;
public class CurrencyServer {
    public static void main(String[] args) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        try {
            CurrencyImpl converter = new CurrencyImpl();
            Naming.rebind("rmi://localhost:1099/CURRENCY", converter);
            System.out.println("El servidor conversor de monedas está listo.");
        } catch (Exception e) {
            System.out.println("Excepción del servidor: " + e);
        }
    }
}

```

Cliente de Conversión de Moneda

```

import java.rmi.Naming;
import java.util.Scanner;
public class CurrencyClient {
    public static void main(String[] args) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        try {
            CurrencyInterface converter = (CurrencyInterface) Naming.lookup("rmi://localhost:1099/CURRENCY");
            System.out.println("Conectado al servidor conversor de monedas");

            Scanner sc = new Scanner(System.in);
            System.out.print("Ingrese monto en soles: ");
            double soles = sc.nextDouble();

            System.out.println("1. Convertir a Dolares");
            System.out.println("2. Convertir a Euros");
            System.out.print("Seleccione opcion: ");
            int opcion = sc.nextInt();

            if (opcion == 1) {
                System.out.printf("Monto en Dolares: %.2f\n", converter.convertirADolares(soles));
            } else if (opcion == 2) {
                System.out.printf("Monto en Euros: %.2f\n", converter.convertirAEuros(soles));
            } else {
                System.out.println("Opcion no valida");
            }
            sc.close();
        } catch (Exception e) {
            System.out.println("Excepción del cliente: " + e);
        }
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

Ejemplo de RMI en Java: Calculadora

Interfaz de Calculadora

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    public int add(int a, int b) throws RemoteException;
    public int sub(int a, int b) throws RemoteException;
    public int mul(int a, int b) throws RemoteException;
    public int div(int a, int b) throws RemoteException;
}
```

Implementación de Calculadora

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {

    public CalculatorImpl() throws RemoteException {
        super();
    }

    @Override
    public int add(int a, int b) throws RemoteException {
        return a + b;
    }

    @Override
    public int sub(int a, int b) throws RemoteException {
        return a - b;
    }

    @Override
    public int mul(int a, int b) throws RemoteException {
        return a * b;
    }

    @Override
    public int div(int a, int b) throws RemoteException {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}
```

Servidor de Calculadora

```
import java.rmi.Naming;

public class CalculatorServer {
    public CalculatorServer() {
        try {
            Calculator c = new CalculatorImpl();
            Naming.rebind("rmi://localhost:1099/CalculatorService", c);
            System.out.println("Calculator Service is ready.");
        } catch (Exception e) {
            System.out.println("Trouble: " + e);
        }
    }

    public static void main(String args[]) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        new CalculatorServer();
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

Cliente de Calculadora

```
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.net.MalformedURLException;
import java.rmi.NotBoundException;

public class CalculatorClient {
    public static void main(String[] args) {
        System.setProperty("java.rmi.server.hostname", "127.0.0.1");
        if (args.length < 2) {
            System.out.println("Usage: java CalculatorClient <num1> <num2>");
            return;
        }
        int num1 = Integer.parseInt(args[0]);
        int num2 = Integer.parseInt(args[1]);

        try {
            Calculator c = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");
            System.out.println("The subtraction of " + num1 + " and " + num2 + " is: " + c.sub(num1, num2));
            System.out.println("The addition of " + num1 + " and " + num2 + " is: " + c.add(num1, num2));
            System.out.println("The multiplication of " + num1 + " and " + num2 + " is: " + c.mul(num1, num2));
            System.out.println("The division of " + num1 + " and " + num2 + " is: " + c.div(num1, num2));
        } catch (MalformedURLException murle) {
            System.out.println("MalformedURLException: " + murle);
        } catch (RemoteException re) {
            System.out.println("RemoteException: " + re);
        } catch (NotBoundException nbe) {
            System.out.println("NotBoundException: " + nbe);
        } catch (ArithmeticException ae) {
            System.out.println("ArithmeticException: " + ae);
        }
    }
}
```